

Interview drills

Questions ROSEN can reasonably ask, the stage of the project that answers each, and the honest answer. Rehearse the *reasoning*, not the wording. Where the true answer is "I don't know yet", say so and follow with how you would find out — that is the answer a team of physicists respects.

For the numbers in table form (all thresholds, all gate results, demo-vs-scale side by side), see `docs/interview-reference.md`. This file is updated through the Stage 6 scale rehearsal — all detection/severity/classification/growth numbers below are the final, 9,600-defect-scale results, not the smaller intermediate runs some of the narratives below start from.

The framing question — "you have no real LSM data, so what is this?"

Real LSM data is proprietary, so I built a physics-inspired forward model — magnetic dipoles at burial depth under a drifting geomagnetic background — and then ran the full lifecycle on it exactly as I would on yours: validation first, background removal, features, grouped splits, calibrated uncertainty, tracking, gated release, a served heat map. The data is a stand-in. The methodology, and the places where I chose to be paranoid, are the real content. I also built in off-pipe interference on purpose, because separating a defect from benign magnetic clutter is the hard part, not fitting the model.

Physics and signal

"What actually generates the signal?" Stress magnetisation — mechanical stress changes local permeability and remanent magnetisation, so a stress-concentration zone perturbs the ambient field. I model it as a point dipole, which is the leading multipole term and right in the far field at ~1.5 m stand-off. It is a crude model for an extended flaw and I would expect the real signature to be closer to a line of dipoles or a dipole sheet.

"Three axes — so you can do gradiometry?" No, and this is worth being precise about. Three axes is one vector magnetometer at one point. Gradiometry needs two spatially separated heads. With a single moving sensor you get the **along-track derivative** dB/ds , which suppresses the slowly varying background but is not common-mode rejection. I added a second head at a 0.5 m vertical baseline to get a real vertical gradient, and I measured whether it helps rather than assumed it: **it doesn't, here**. Detection contrast is 3.21x for a single detrended head versus 1.45x for the vertical-gradient difference. Subtracting cancels the background, but it also adds the noise of both heads together, and after detrending has already removed most of the background, that noise cost isn't worth paying. I'd expect gradiometry to win where the background is strongly non-uniform along-track — not the case here. Worth adding: my first version of this measurement was wrong for a boring reason — I'd built the second head from the same background arrays as the first, so common-mode rejection

was perfect by construction, not because the technique worked. I only trusted the 1.45x number once I'd fixed that and the two heads had genuinely independent background noise.

"How do you remove the background?" The defect residual is ~25 nT on a ~45 000 nT field — 0.05%. Two stages: a robust degree-3 polynomial per axis, then a 40 m rolling-median high-pass for whatever the polynomial can't fit. The window has to be long compared with a defect footprint (a few metres at 1.5 m stand-off) and short compared with geomagnetic drift. Choosing that scale separation is the engineering.

I didn't just assume that generalises — I checked it against a real background. My synthetic background is a drift plus one sine wave, which is polynomial-like by construction, so a bare polynomial fit it almost perfectly (residual landed exactly at my 5.0 nT noise floor). That's my detrender flattering itself, not evidence it works. So I pulled two real days of USGS magnetic observatory data — a quiet day and the May 2024 storm, the largest in 20 years — and ran the same detrend against real geomagnetic structure. A bare polynomial does noticeably worse on the real storm data (6.9–11.3 nT residual). My actual two-stage method held up: 7.8–8.1 nT residual and 3.0–3.2x detection contrast across the synthetic, quiet, and storm backgrounds. It survived contact with real physics, and I only know that because I went looking for a way to prove myself wrong.

"How do you tell a defect from a fence post?" Amplitude alone will not do it — an off-pipe source can be strong. The discriminators are geometric: dipole fields fall as $1/r^3$, so an off-centreline source is both weaker *and* broader, with a slower decay in the along-track profile. So I use peak width (FWHM), asymmetry and a fitted decay exponent, not just peak height. And I keep `interference` as an explicit class with its own reported precision, rather than letting it hide inside "negative".

Data and scale

"We have millions of rows from many years — does this scale?" The demo uses SQLite, which is the right tool at 10^7 rows read-mostly with a composite primary key on `(survey_id, chainage_m)`. It stops being right at concurrent multi-writer ingest. The production shape is columnar: Parquet on S3 partitioned by line and run, queried with DuckDB or Athena, with a relational store kept only for the small high-value tables — survey registry, ground truth, DQ reports, indications, model versions. Signal is append-only and immutable; metadata is relational and mutable. I built a scale rehearsal into the plan (~40 lines, $\sim 10^7$ rows) specifically so I am not guessing about that.

"Multiple data sources have to be integrated." The join key is **chainage plus a line identifier**, and the hard part is that every source has its own chainage datum — ILLI odometer, GPS-derived, as-built drawings — so they disagree by metres. I would treat alignment as a first-class step with its own validation: cross-correlate a common feature (girth welds are the natural fiducial), estimate a per-segment offset and stretch, and record the residual alignment error as a data-quality metric that propagates into localisation uncertainty. Fusing before you have quantified the misalignment is how you manufacture false correlations.

"How do you handle duplicates? You'll get the same survey twice." Two hashes with different jobs. A hash of the file bytes proves provenance and immutability, but it gives false negatives for deduplication — re-export the same survey with different float formatting and you get a new hash for identical data. So there is also a

canonical content hash over normalised content: contract columns in declared order, sorted by sample index, cast to contract dtypes and rounded to a declared precision. Ingest is idempotent on that, and a *different* content hash for an existing (line, run) is an error rather than an overwrite — raw data is immutable. The case neither hash catches is **partial overlap**: two surveys of the same line that intersect, from a re-run over a bad section or a survey split across files. That needs an interval check plus signal cross-correlation in the overlap — and it matters beyond tidiness, because an undetected overlap landing in two different folds is leakage. It's why my group key is (line, block) and never block alone.

"What's in your data contract?" Dtypes chosen from the physics, not habit. Latitude and longitude must be float64 — float32 ULP at 47° N is about 0.42 m, comparable to my 0.5 m sample spacing, so float32 GPS would quantise the survey. Field values can be float32, since ULP at 45 000 nT is about 0.004 nT, below any magnetometer's resolution — but the arithmetic runs in float64, because detrend least-squares fits and cumulative window sums accumulate error that storage precision says nothing about. And the physical key is an integer sample index, never the float chainage: $0.5 * 3$ is not 1.5 in binary, and a float join key silently drops rows between tables built in different code paths.

"Ground truth?" Scarce, delayed and biased — you only dig where you predicted, so your labels are censored by your own past ranking. That means naive precision on dug sites overstates performance, and it argues for a small randomised or uncertainty-driven exploration budget in the dig programme. That is the active-learning stretch goal, and it is a genuine business argument, not a research nicety.

Modelling

"Why not just threshold?" I do — MAD threshold is the baseline, and any candidate has to beat it by a stated margin, on the confidence interval, not the point estimate. I measured this four times, at increasing scale, and the honest final answer is: **it doesn't beat baseline, and I now know that with real confidence, not a shrug.**

1. First, 12 defects: gap -0.056, CI [-0.167, 0.056] — too little data to say anything.
2. Scaled to 60 defects (5 independent lines, not more defects crammed onto one line, which I tried first and it degraded a different gate by contaminating background rows with neighbouring sources' field tails): gap -0.028, CI [-0.083, 0.022] — narrower, still straddling zero, consistent with a small real effect.
3. I ran an EDA over the 46-feature set, found a specific, fixable mechanism (below), fixed it, and re-ran: gap closed to **-0.006, CI [-0.067, 0.050]**. At the time this looked like the fix had worked and the two models were now roughly at parity.
4. That reading was wrong, and I only found out because I didn't stop there. I scaled the same, already-fixed pipeline to 9,600 defects (~40 independent lines, ~9.6M rows) purely to get a confident answer. Result: **gap -0.029, CI [-0.033, -0.026]** — a tight interval that excludes zero, landing almost exactly back at step 2's number. The apparent fix at step 3 was itself a small-sample coincidence, not a real improvement that held up at scale.

So the current, honest conclusion is: IsolationForest is genuinely, reproducibly, if only slightly, worse than a robust MAD threshold on this corpus. Not a shrug, not "still not enough data" — a real, small, negative effect,

confirmed at 800x the original scale, with a mechanism I can point to. The lesson from steps 3-to-4 specifically: don't treat an interval that only just crosses zero as proof a fix worked — confirm it at a scale that can actually tell "fixed" apart from "noise."

Where exactly it loses, from the same Stage 6 comparison (both models get the same dig-budget of holes, so their misses are directly comparable):

Share of the dig budget	MAD	IsolationForest
Wasted on real interference	23.3%	23.0%
Wasted on empty ground	0.0%	3.6%
Average position error	0.28 m	0.42 m

MAD's mistakes are all the "designed" kind — real interference sources, which is the false-positive trap I built on purpose. IsolationForest makes that same mistake at almost the same rate, but adds a new one: 3.6% of its digs land on nothing at all, and it localises defects roughly 50% worse. Its one small edge on interference (0.3 points) is about ten times smaller than what it loses elsewhere.

Why, mechanically — one shallow reason, one structural.

Shallow: IsolationForest picks its split feature uniformly at random, and about 20 of my 46 features are amplitude-based and mutually correlated — essentially the same signal MAD already thresholds on (two of them were *exact* duplicates under the current config, a real bug I found via EDA and fixed with a `feature_version` bump). So most splits re-derive MAD's own decision, with added variance from random feature selection, rather than using the handful of shape features (`w25m_kurt` , `w25m_zcr` , `w10m_kurt`) that actually separate a defect from interference. I tried fixing this directly — an `emphasis_repeats` knob that tiles those three features in the fitted matrix to raise their selection odds — and it's what produced step 3's apparent improvement. It didn't survive scaling.

Structural, and the more important one: I built interference to have the *same amplitude* as a defect on purpose — that's the whole point of the false-positive trap. So to an outlier-detection method, a defect and a piece of buried junk are equally unusual; being unusual is the property they share, not the property that tells them apart. Only shape does that, and shape is exactly the information an unsupervised, amplitude-heavy method is least equipped to use. I know the information is genuinely present in the features, because my supervised classifier, seeing the exact same feature set with labels, separates interference from everything else at **99.9% precision**. The ceiling on an unsupervised fix here is real, and it's a framing problem, not a tuning problem.

"Why two stages?" Because the unit of decision is an **indication**, not a 0.5 m sample. Row-level scoring, then peak clustering into indications, then per-indication severity and class. It also removes an evaluation trap: `severity_smys` is zero off-defect and constant inside the label window, so a row-level regressor scores beautifully by predicting zero everywhere.

"How do you split?" By line, or by 100 m chainage block, with `GroupKFold` — never randomly. Adjacent 0.5 m samples are near-duplicates and the same defect recurs in every survey, so a random split leaks in two

directions at once and produces a number I would not be able to defend. There is a CI test that fails if any group crosses folds.

"Which metric?" Not accuracy, not ROC-AUC — positives are under 3% of rows. PR-AUC as a diagnostic, and as the headline **recall at a fixed dig budget**: if you fund five digs per kilometre, what fraction of real defects do you find, and how many empty holes do you dig? That maps directly onto what an integrity engineer is optimising.

"Uncertainty?" LightGBM quantile regression for the shape, wrapped in split conformal for a distribution-free 90% interval with a finite-sample coverage guarantee — under exchangeability. A new line or a new scanner breaks exchangeability, which is why coverage is monitored in production and re-fit on new calibration data rather than assumed. First real run (12-defect corpus) measured 0.727 against the 0.87-0.93 target band — below it, though the CI was [0.485, 0.939], containing the target, so at $n=11$ matched defects the honest statement was "can't confirm calibration is good, can't rule it out either," not "calibration is broken." I'd also found a real bug in my own conformal code on that first run: a plain (1-alpha) quantile for the calibration margin instead of the finite-sample- corrected level the theory (Romano et al. 2019) specifies, which systematically undercovers on small calibration sets. Fixed it, verified with unit tests that hand-check the correction against a naive quantile. On a 5-line/~60-defect corpus ($n=137$ matched severity indications, same scale-up that sharpened the Stage 3 measurement above), coverage came in at **0.920 [0.867, 0.967]** — inside the target band, gate passes — with MAE roughly halved versus the global-mean baseline (7.46 vs 15.09 nT). Same conformal code both times; more calibration points is what actually closed the gap, which is the expected story for a distribution-free method under a genuine small-sample regime, not a coincidence. Still conditional on Stage 3's detector, which doesn't beat baseline — a passing severity gate validates the CQR methodology, not the whole pipeline's choice of anomaly model.

Scaling further, to Stage 6's 9,600 defects, sharpened it again: coverage **0.896 [0.891, 0.901]**, comfortably inside [0.87, 0.93], and MAE dropped to **3.36 nT** against a 15.0 nT baseline — better than the smaller-scale run, as expected with more calibration data.

That run also surfaced a real bug worth being upfront about, because it's a good example of a failure mode no data check alone would catch. My generator sets severity to NaN off-defect. In 36 of 18,917 matched indications (0.4%) a detected peak landed just outside a defect's exact label window while still inside the looser dig-matching tolerance — so it got credited as a match, but its true severity was NaN. One NaN reaching the conformal calibration step silently poisoned the *entire fold's* margin, because the quantile function that computes it propagates NaN. A 0.4% data issue turned into 0% coverage for that whole fold — a cliff, not a gradual degradation. Fixed by dropping NaN-labelled rows before calibration, with a second guard inside the model itself. I only found it because I checked coverage at scale rather than trusting that a demo-scale pass meant the mechanism was sound.

"How do you decide defect type — SCC vs weld vs dent vs corrosion?" Multiclass LightGBM (isotonic-calibrated $P(\text{defect})$, degenerate-fold fallbacks for the small per-class counts, a physics-consistency SHAP check that denylists absolute chainage) — that machinery is real and built. But the honest number, from my real run: recall is scc 0.125, weld 0.111, dent 0.646, corrosion 0.383, versus **interference at 0.778 recall / 0.909**

precision. I didn't just report that gap, I checked why. In `generate.py`, each synthetic defect's `type` is assigned `rng.choice(["scc", "weld", "dent", "corrosion"])` at generation time, and grepping the file confirms that value is read in exactly one place afterward — writing the ground-truth label column. It never reaches the dipole model: amplitude, orientation and decay all come from `severity / orientation`, drawn independently of `type`. So in this synthetic dataset the four defect subtypes are **physically indistinguishable in the sensor data by construction** — there is no signal in the features for a classifier to find, and scc/weld sitting at or below the 25% random-guess floor for a 4-way choice is the model doing exactly what the data supports, not underperforming. Interference is a genuinely different physical source (bigger magnetic moment, broader/off-pipe geometry — see the interference section above), which is why the model separates it well and can't separate the other four. I'd rather say that plainly than let a strong interference number imply the subtype numbers are equally real. Fixing it for real means extending the generator so each defect type carries a distinguishing physical signature (the decay/shape differences a metallurgist would actually expect between SCC, a weld anomaly, a dent, and corrosion) — a stated, scoped data gap, not a modelling failure, and the same "add its baseline first, report the honest number" discipline as everything else in this project.

One more thing worth knowing before an interview, because on the surface it looks like it contradicts the paragraph above: at Stage 6 scale (9,600 defects), SCC recall rose to **0.636 [0.620, 0.652]** — tight, and well above the ~25% you'd expect from a genuinely blind 4-way guess. My first instinct was that more data let the model find something real. So I checked directly, on the actual generated files, rather than trusting the number: at true population scale (tens of thousands of defects per type), severity, `w25m_kurt`, `fwhm_m`, `r_mag_nt` and `decay_exponent` are all statistically identical across scc/weld/dent/corrosion — the small differences visible at demo scale (only ~15 defects per type) were sampling noise that washes out at scale, exactly as the "physically indistinguishable by construction" argument predicts. So the 0.636 recall almost certainly doesn't reflect real physical separation in the features. I don't have a full explanation for it yet — it needs a look at per-class precision and the confusion matrix, which this project hasn't captured — and I would say exactly that if asked, rather than present 0.636 as evidence the classifier works. A number that looks good deserves the same scrutiny as one that looks bad.

"Deep learning?" A 1-D CNN over the residual window is a reasonable next step and I have it planned, but gradient boosting on well-designed physics features is the right first model at this data scale, it trains in seconds on 16 cores, and it is far easier to interrogate with SHAP. I would move to a CNN when I have enough labelled indications that learned filters beat hand-designed shape features — and I would keep the boosted model as the challenger baseline, not delete it.

"Growth and remaining life?" Three surveys with ~15% growth per survey. Fitting twelve independent growth curves off three points each would be over-fitting, so I use partial pooling — each defect's own rate is shrunk toward a population rate, more so with fewer observations — then project to a limit state for remaining life, propagating the severity interval through so the output is a range, not a date. The gate compares against a no-growth baseline on one-step-ahead severity error, not on remaining life directly, since an unchanging severity implies infinite life, which has no error to compare against a finite one.

It passes, and I want to be precise about what that does and doesn't prove. My generator grows every defect by exactly 15% per survey, with no noise in that growth law. A correctly built pooled estimator should recover

$\ln(1.15) = 0.1398$ almost exactly — and it does: measured population rate is 0.1398. That's a strong confirmation the *estimator* is implemented correctly. It is not evidence the method works on real defect growth, which is noisy and uneven, not geometric. I say that plainly in the model card next to the number: with as few as three observations per defect, per-defect rates are almost entirely shrunk to the population value, so every remaining-life figure is provisional, not a calibrated forecast.

MLOps

"How is a run reproducible?" `git_sha` + `config_sha256` + `data_sha256`, all three logged to MLflow and written into the `model_run` table, one `config.yaml`, one seed, and a determinism test in CI that re-runs training and compares metrics. One practical catch: LightGBM is not bit-reproducible across thread counts by default, so `deterministic=True`, `force_row_wise=True` and a fixed `num_threads` are set from day one — otherwise that test flakes intermittently, which is worse than not having it.

"What happens when a survey fails validation?" It goes to a quarantine bucket with its DQ report attached and raises a ticket; it is never deleted and never silently skipped. In CI the same failure exits non-zero, but in production the run continues to the next survey. A pipeline that halts the whole nightly job because one survey had a stuck channel gets switched off by its operators within a month, and then you have no validation at all.

"You changed the feature code — what happens to everything you already computed?" This is the version people forget, and it is the nastiest, because it is training/serving skew that no *data* check can detect — the data is fine, the code moved. So `feature_version` is explicit: it is in the feature-store path, it is in the model bundle, and a bundle refuses to load against a mismatch. Bumping it forces a documented decision — backfill, or pin old models to old features. Both are fine; leaving it implicit is not.

"How confident are you in that metric?" Not very, and I report that rather than hiding it. Every headline number carries a bootstrap confidence interval resampled over *groups* — lines and defects — not rows. With twelve defects, "recall 0.83" is a point estimate on a sample of twelve, and the interval is wide. The promotion gate compares intervals, not point estimates, precisely so it does not promote a model on sampling variation.

"Training vs inference — what stops them diverging?" They share the code path. The same validator runs at ingest and inside `predict`. Features are split into per-survey transforms, which are legitimately re-fit at inference because they *are* background removal, and fitted transforms — scalers, calibration, conformal quantiles, threshold — which live inside the versioned bundle and are never recomputed. The bundle stores its feature list in order and refuses to load against a mismatched feature set.

"How does a model reach production?" CI gates: DQ pass, leakage and determinism tests green, primary metric beats the incumbent on the same holdout *compared on intervals*, conformal coverage inside its band, no leaky feature in the top-10 SHAP list, bundle round-trip exact. Passing cuts a `pipeline_release` and sets the `@challenger` alias automatically; it then runs in shadow, and moving `@champion` is a human decision with a recorded reason. Deployment pins the `pipeline_version`; rollback repoints the alias, and bundles are immutable so a rollback target always exists. One caveat I'd raise unprompted: comparing every candidate

against one fixed holdout eventually selects on holdout noise, so the final test set is touched rarely and its uses are counted.

"How would you process our archive?" That is the workload the whole design is centred on — backfill, not steady-state scoring. It's a Dagster asset graph partitioned by survey, so processing the archive is a partition backfill rather than a bespoke script, and a failure at survey 4,000 of 6,000 costs one partition rather than the run. Retries are 3× on IO and **zero on a data-quality failure**, because a bad survey isn't a flake — retrying it three times just delays the ticket. I chose Dagster over Airflow specifically because its asset code-version model maps onto my `feature_version`, so bumping the feature code re-materialises exactly the affected partitions.

"Where does a prediction actually get served?" The serving layer is the batch scoring job that writes the indication table and the GeoJSON the GIS system consumes — versioned output schema, a consumer contract test in CI, idempotent on (survey, pipeline version), with a 24-hour freshness target. The Streamlit app is a *consumer* of that table, not the server. I'd be careful about that distinction because calling a dashboard "the serving layer" hides the fact that nobody has defined the actual contract.

"How do you validate a new model when the ground truth is six months away?" You can't, on outcomes — so the only pre-label evidence is **shadow scoring**. Champion and challenger both score every live survey, the challenger's output is written flagged as shadow, and I analyse *disagreement*: change in indications per kilometre, rank churn at the dig budget, and whether the disagreements cluster in surveys that already had data-quality warnings. It gives you signal on day one instead of month six. Then when labels do arrive, I evaluate by vintage — pin a `truth_as_of`, reconstruct what was known on that date, score forward — so a label correction can't retroactively improve a historical result.

"What's your deployable unit?" Not a model — a `pipeline_version` manifest pinning four model versions, the feature version, the schema version and the container digest. An indication is produced by all of them, so its lineage is only answerable if they're pinned together, and they have to be promoted and rolled back together. Rollback is repointing the `@champion` alias, and I drill it in CI, because the usual way rollback fails is that the old bundle no longer loads against current code — which my own feature-version guard would cause. So the policy is that the last three feature versions stay loadable and CI proves it.

"What breaks in production and how do you know?" Drift, in three flavours: feature drift (PSI vs the bundle's training reference), prediction drift (indications per km), and background regime shift (median field and drift slope vs that line's history). Plus measured conformal coverage as verification results arrive. The failure I would most expect is a new scanner unit or recalibration shifting the background — a data problem that looks exactly like a model problem if you are not monitoring the inputs.

Process, honesty, and what was new to you

Questions in this section aren't about the domain — they're about how you actually worked, and in a lot of interviews they matter more than the domain answers because they're the ones that transfer to a role that isn't this exact project. Answer with your own true specifics, not a generic "I learned a lot about MLOps" — name the actual thing.

"Tell me about something in this project you hadn't used before." Be concrete: which technique, when you ran into needing it, what you did to get up to speed (read the original paper, worked a toy example by hand, implemented it and tested it against a naive baseline before trusting it), and what surprised you once you had it running. Naming a specific gap and how you closed it is a stronger answer than implying you walked in knowing everything — this project genuinely has two candidates for this: IsolationForest, and split conformal prediction (CQR). If either was new to you, say so plainly and say what studying it involved.

"Walk me through something that didn't work." Four real ones, fully reportable without hedging: 1. IsolationForest vs the MAD baseline (Stage 3) — never beat it, across four real runs at increasing scale. First (12 defects): recall gap -0.056, CI [-0.167, 0.056] — too little data to say anything. Scaled to 60 defects (5 independent lines, not more defects packed onto one line — that move alone surfaced a real data-generation bug I fixed first): gap -0.028, CI [-0.083, 0.022] — tighter, consistent with a real small effect. I then ran a real EDA, found the model was fit on ~20 redundant amplitude features drowning out the 3 that actually separate defect from interference, fixed it (dropped 2 exact-duplicate features, added a knob to up-weight the other 3), and re-ran: gap closed to -0.006, CI [-0.067, 0.050]. That looked like a fix. It wasn't: scaling the same, already-fixed pipeline to 9,600 defects gave gap -0.029, CI [-0.033, -0.026] — a confident, real, negative result, landing almost exactly back at the pre-fix number. The honest end state is "genuinely, slightly worse than baseline, confirmed at 800x scale," not "parity." 2. Split-conformal undercoverage (Stage 4) — traced to a real bug in my own code (a naive quantile where the theory calls for a finite-sample-corrected one), fixed it, and the gate *still* didn't clear on the original 12-defect data (0.727 vs a [0.87,0.93] target, CI containing the target band). Left it there rather than manufacture a happy ending — until a 5x corpus scale-up gave conformal enough calibration points to actually pass, and scaling further to 9,600 defects held that pass (0.896) with MAE dropping to 3.36 nT. Worth being clear that the code fix was necessary but not sufficient by itself; scale is what closed the gap. 3. The vertical gradiometer (Stage 2.5) — the standard fix for this kind of background problem, and it made things worse, not better: 1.45x detection contrast versus 3.21x for a single detrended head. Subtracting two heads cancels the background but adds both heads' noise, and after detrending has already removed most of the background, that noise cost isn't worth paying here. My first version of this measurement was actually a second bug layered on top: I'd built the second head from the same background arrays as the first, so common-mode rejection was perfect by construction. I only trusted the negative result once I'd fixed that. 4. A 0.4% data issue that caused a 0% result (Stage 6) — 36 of 18,917 matched severity indications (0.4%) had a NaN true severity from a peak landing just outside a defect's label window. One NaN reached the conformal calibration step and silently poisoned that entire fold's margin — 0.4% incidence, 0% coverage for the whole fold, a cliff rather than a gradual degradation. Fixed by dropping NaN rows before calibration, with a second guard in the model itself. Only found because I checked the actual coverage number at scale rather than assuming a demo-scale pass meant the mechanism was sound everywhere.

"How do you know your negative results aren't just bugs?" Because I kept checking after the number looked better, not just when it looked bad. The conformal undercoverage was traced to a specific formula error, confirmed by hand-computing the correction on a small example. The IsolationForest gap is the sharper example: at 60 defects, after a real EDA-driven fix, it looked resolved — gap -0.006, CI crossing zero. I could have stopped there and called it fixed. Instead I scaled the same pipeline to 9,600 defects specifically to get a confident answer, and the "fix" reverted: gap -0.029, CI excluding zero, landing almost exactly back at the pre-fix

estimate. That's the actual discipline — a result that only just crosses zero at small scale isn't confirmed until you've checked it at a scale that could actually tell fixed from noise, and I got the wrong answer once by not doing that soon enough.

"What would you do next with more time or data?" I already took that lever as far as it goes — Stage 6 scaled the corpus to 9,600 defects (~800x), and it gave a definitive rather than a hopeful answer: severity's gate, which really was data-starved, now passes cleanly (0.896 coverage); detection's gate, which I'd hoped was also just data-starved, turned out not to be — the gap is real and confirmed, not resolved. So more of the same lever is spent. What is actually left: giving the four defect subtypes (SCC/weld/dent/corrosion) a real, distinguishing physical signature in the generator, since right now they are — correctly — indistinguishable by design, which caps what classification can ever achieve here; a supervised alternative to IsolationForest now that I know unsupervised amplitude methods have a real ceiling on this task; and the deploy/shadow-scoring infrastructure that's designed in `docs/production-architecture.md` but not built, because there's no cloud account to point it at.

Questions to ask them

- How is chainage reconciled between LSM, ILI and as-built records today, and what is the residual alignment error?
- What fraction of LSM indications get excavated, and is any of the dig budget randomised?
- Is stand-off distance measured per survey or assumed from as-built depth of cover?
- Do you have paired LSM and ILI runs on the same line — that is the supervised dataset.
- What is the current false-dig rate, and what would a useful improvement look like in euros?
- Where does the model output actually land — a report, a GIS layer, an integrity management system?